```
%config Completer.use_jedi = False

%matplotlib inline
!pip show tensorflow
!wget -cq https://ti.arc.nasa.gov/c/5 -O naza.zip
!unzip -qqo naza.zip -d battery_data
```

```
Name: tensorflow
Version: 2.10.0
Summary: TensorFlow is an open source machine learning framework for
everyone.
Home-page: https://www.tensorflow.org/
Author: Google Inc.
Author-email: packages@tensorflow.org
License: Apache 2.0
Location: c:\users\kamal\anaconda3\envs\ncmapss_gpu\lib\site-packages
Requires: absl-py, astunparse, flatbuffers, gast, google-pasta,
grpcio, h5py, keras, keras-preprocessing, libclang, numpy, opt-einsum,
packaging, protobuf, setuptools, six, tensorboard, tensorflow-
estimator, tensorflow-io-gcs-filesystem, termcolor, typing-extensions,
wrapt
Required-by:

'wget' is not recognized as an internal or external command,
operable program or batch file.
'unzip' is not recognized as an internal or external command,
operable program or batch file.
```

```
import datetime
import numpy as np
import pandas as pd
from scipy.io import loadmat
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from sklearn import metrics
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.io import loadmat
```

For the Deep Learning model proposed in [1] it is only necessary to collect the data related to the discharge of the battery, for this a function is created in Python that is in charge of reading this data from the ".mat" file and storing it in memory in two pandas DataFrame for later access. After loading the dataset, a description of the data is made using panda functions to verify if the data loading was correct.

```
def load_data(battery):
  mat =mat =
loadmat('/Users/Kamal/Documents/AUC/thesiswork/Nasa_battery_dataset/'+
battery + '.mat')
  print('Total data in dataset: ', len(mat[battery][0, 0]['cycle']
```

```
[0]))
  counter = 0
  dataset = []
  capacity_data = []

  for i in range(len(mat[battery][0, 0]['cycle'][0])):
    row = mat[battery][0, 0]['cycle'][0, i]
    if row['type'][0] == 'discharge':
      ambient_temperature = row['ambient_temperature'][0][0]
      date_time = datetime.datetime(int(row['time'][0][0]),
                                    int(row['time'][0][1]),
                                    int(row['time'][0][2]),
                                    int(row['time'][0][3]),
                                    int(row['time'][0][4])) +
datetime.timedelta(seconds=int(row['time'][0][5]))
      data = row['data']
      capacity = data[0][0]['Capacity'][0][0]
      for j in range(len(data[0][0]['Voltage_measured'][0])):
        voltage_measured = data[0][0]['Voltage_measured'][0][j]
        current_measured = data[0][0]['Current_measured'][0][j]
        temperature_measured = data[0][0]['Temperature_measured'][0]
[j]
        current_load = data[0][0]['Current_load'][0][j]
        voltage_load = data[0][0]['Voltage_load'][0][j]
        time = data[0][0]['Time'][0][j]
        dataset.append([counter + 1, ambient_temperature, date_time,
capacity,
                        voltage_measured, current_measured,
                        temperature_measured, current_load,
                        voltage_load, time])
      capacity_data.append([counter + 1, ambient_temperature,
date_time, capacity])
      counter = counter + 1
  print(dataset[0])
  return [pd.DataFrame(data=dataset,
                       columns=['cycle', 'ambient_temperature',
'datetime',
                                'capacity', 'voltage_measured',
                                'current_measured',
'temperature_measured',
                                'current_load', 'voltage_load',
'time']),
          pd.DataFrame(data=capacity_data,
                       columns=['cycle', 'ambient_temperature',
'datetime',
                                'capacity'])]
dataset, capacity = load_data('B0005')
pd.set_option('display.max_columns', 10)
```

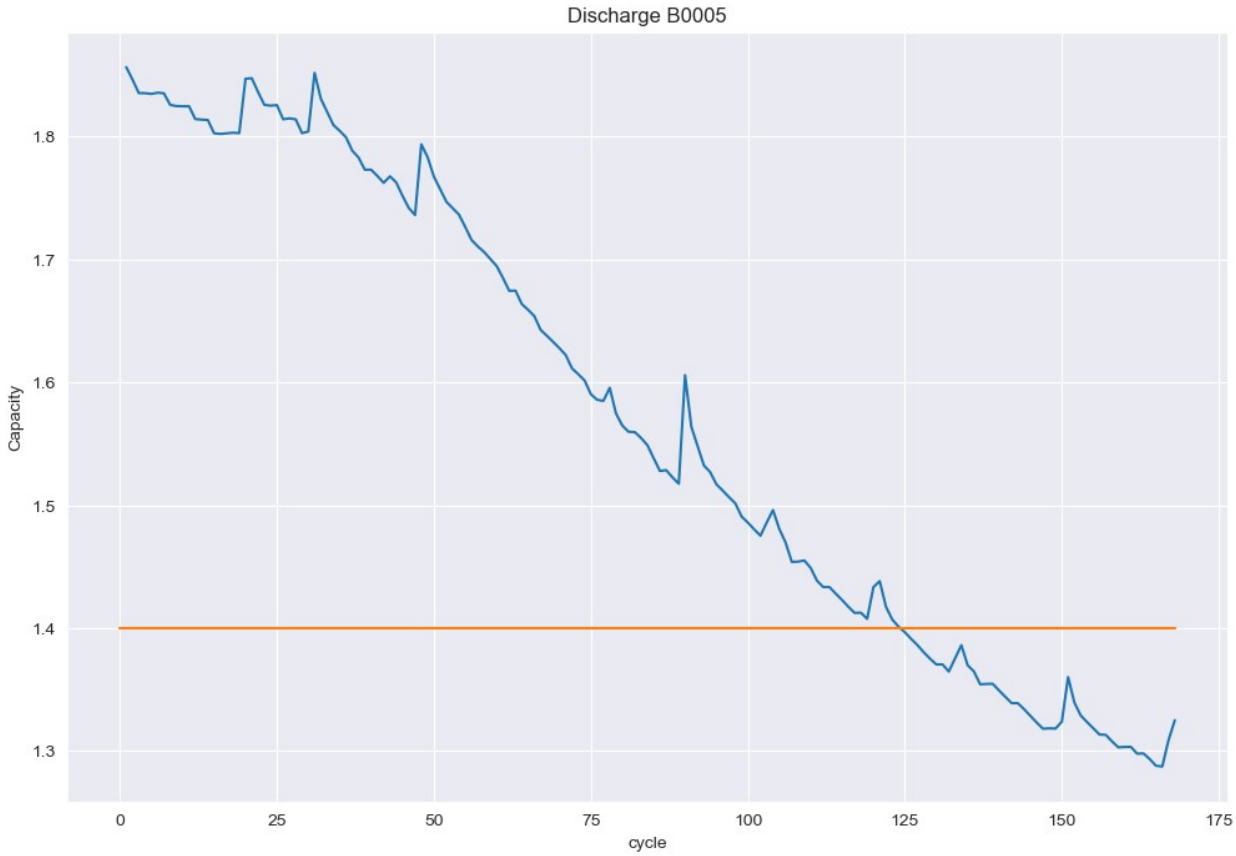
```
print(dataset.head())
dataset.describe()

Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 1.8564874208181574,
4.191491807505295, -0.004901589207462691, 24.330033885570543, -0.0006,
0.0, 0.0]
   cycle  ambient_temperature            datetime  capacity
voltage_measured  \
0      1                   24 2008-04-02 15:25:41  1.856487
4.191492
1      1                   24 2008-04-02 15:25:41  1.856487
4.190749
2      1                   24 2008-04-02 15:25:41  1.856487
3.974871
3      1                   24 2008-04-02 15:25:41  1.856487
3.951717
4      1                   24 2008-04-02 15:25:41  1.856487
3.934352

   current_measured  temperature_measured  current_load  voltage_load
time
0         -0.004902             24.330034       -0.0006         0.000
0.000
1         -0.001478             24.325993       -0.0006         4.206
16.781
2         -2.012528             24.389085       -1.9982         3.062
35.703
3         -2.013979             24.544752       -1.9982         3.030
53.781
4         -2.011144             24.731385       -1.9982         3.011
71.922

              cycle  ambient_temperature      capacity
voltage_measured  \
count  50285.000000              50285.0  50285.000000
50285.000000
mean      88.125942                 24.0      1.560345
3.515268
std       45.699687                  0.0      0.182380
0.231778
min        1.000000                 24.0      1.287453
2.455679
25%       50.000000                 24.0      1.386229
3.399384
50%       88.000000                 24.0      1.538237
3.511664
75%      127.000000                 24.0      1.746871
3.660903
max      168.000000                 24.0      1.856487
```

```
4.222920

       current_measured  temperature_measured  current_load
voltage_load  \
count      50285.000000          50285.000000  50285.000000
50285.000000
mean          -1.806032             32.816991      1.362700
2.308406
std            0.610502              3.987515      1.313698
0.800300
min           -2.029098             23.214802     -1.998400
0.000000
25%           -2.013415             30.019392      1.998000
2.388000
50%           -2.012312             32.828944      1.998200
2.533000
75%           -2.011052             35.920887      1.998200
2.690000
max            0.007496             41.450232      1.998400
4.238000

                time
count  50285.000000
mean    1546.208924
std      906.640295
min        0.000000
25%      768.563000
50%     1537.031000
75%     2305.984000
max     3690.234000
```

The following graph shows the aging process of the battery as the charge cycles progress. The horizontal line represents the threshold related to what can be considered the end of the battery's life cycle.

```
plot_df = capacity.loc[(capacity['cycle']>=1),['cycle','capacity']]
sns.set_style("darkgrid")
plt.figure(figsize=(12, 8))
plt.plot(plot_df['cycle'], plot_df['capacity'])
#Draw threshold
plt.plot([0.,len(capacity)], [1.4, 1.4])
plt.ylabel('Capacity')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.title('Discharge B0005')
```

```
Text(0.5, 1.0, 'Discharge B0005')
```



It is also necessary to calculate the SoH of the battery, since this is the data that will be predicted using the * deep learning * model.

```
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity[attrib]
C = dis_ele['capacity'][0]
for i in range(len(dis_ele)):
    dis_ele['SoH']=(dis_ele['capacity'])/C
print(dis_ele.head(5))
```

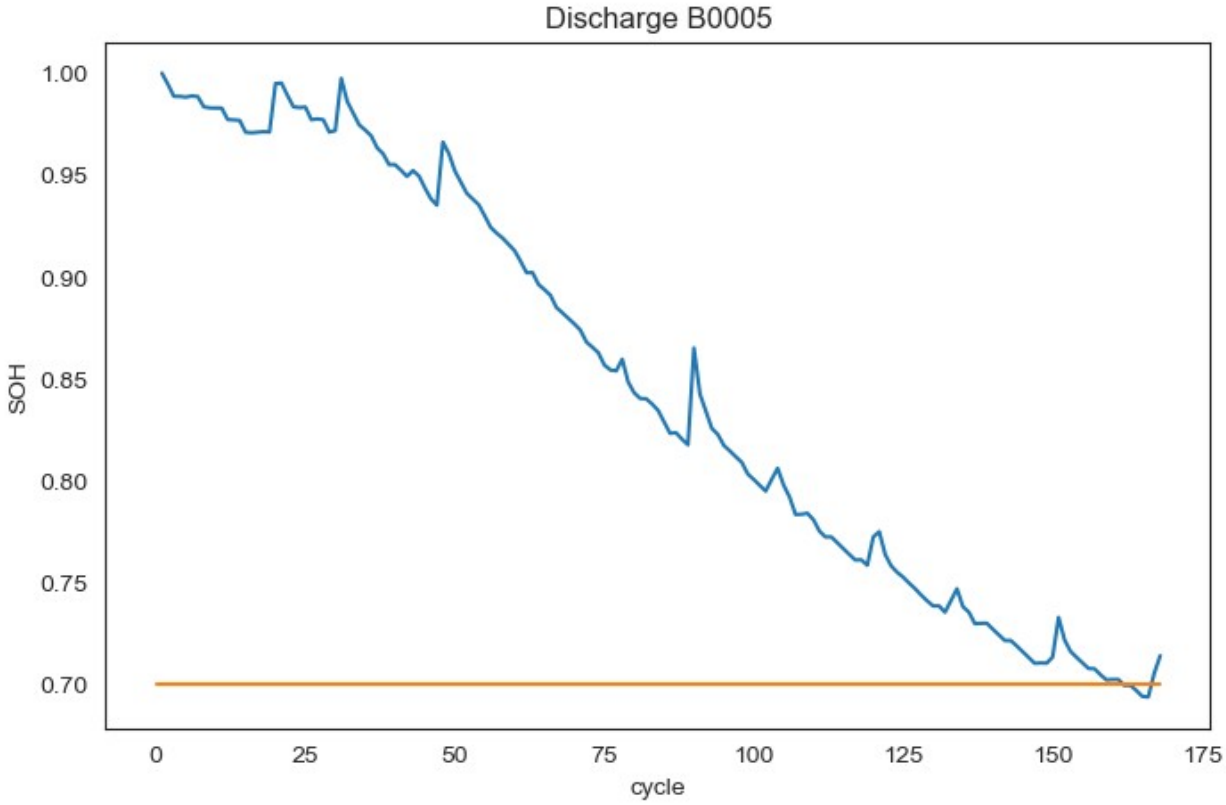
```
   cycle            datetime  capacity       SoH
0      1 2008-04-02 15:25:41  1.856487  1.000000
1      2 2008-04-02 19:43:48  1.846327  0.994527
2      3 2008-04-03 00:01:06  1.835349  0.988614
3      4 2008-04-03 04:16:37  1.835263  0.988567
4      5 2008-04-03 08:33:25  1.834646  0.988235
```

Similarly to what has been done previously, a graph of the SoH is made for each cycle, the horizontal line represents the threshold of 70% in which the battery already fulfills its life cycle and it is advisable to make the change.

```
plot_df = dis_ele.loc[(dis_ele['cycle']>=1),['cycle','SoH']]
sns.set_style("white")
```

```
plt.figure(figsize=(8, 5))
plt.plot(plot_df['cycle'], plot_df['SoH'])
#Draw threshold
plt.plot([0.,len(capacity)], [0.70, 0.70])
plt.ylabel('SOH')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.title('Discharge B0005')
```

```
Text(0.5, 1.0, 'Discharge B0005')
```




```
C = dataset['capacity'][0]
soh = []
for i in range(len(dataset)):
  soh.append([dataset['capacity'][i] / C])
soh = pd.DataFrame(data=soh, columns=['SoH'])

attribs=['capacity', 'voltage_measured', 'current_measured',
         'temperature_measured', 'current_load', 'voltage_load',
'time']
train_dataset = dataset[attribs]
sc = MinMaxScaler(feature_range=(0,1))
train_dataset = sc.fit_transform(train_dataset)
```

```
print(train_dataset.shape)
print(soh.shape)
```

```
(50285, 7)
(50285, 1)
```

Preparation of the model, 3 dense layers are used, and the parameters are used as they are in the paper: 3 dense layers and one dropout, and one of the ADAM type is used as optimizer

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import LSTM
from tensorflow.keras.optimizers import Adam

model = Sequential()
model.add(Dense(8, activation='relu',
input_dim=train_dataset.shape[1]))
model.add(Dense(8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dropout(rate=0.25))
model.add(Dense(1))
model.summary()
model.compile(optimizer=Adam(beta_1=0.9, beta_2=0.999, epsilon=1e-08),
loss='mean_absolute_error')
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 dense_4 (Dense)              (None, 8)                 64

 dense_5 (Dense)              (None, 8)                 72

 dense_6 (Dense)              (None, 8)                 72

 dropout_1 (Dropout)          (None, 8)                 0

 dense_7 (Dense)              (None, 1)                 9

=================================================================
Total params: 217
Trainable params: 217
Non-trainable params: 0
_________________________________________________________________
```

The model is trained, 50 epochs are used for training

```
model.fit(x=train_dataset, y=soh.to_numpy(), batch_size=25, epochs=30)

Epoch 1/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.1133
Epoch 2/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0337
Epoch 3/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0338
Epoch 4/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0330
Epoch 5/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0332
Epoch 6/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0330
Epoch 7/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0328
Epoch 8/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0329
Epoch 9/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0328
Epoch 10/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0325
Epoch 11/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0329
Epoch 12/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0326
Epoch 13/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0326
Epoch 14/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0328
Epoch 15/30
2012/2012 [==============================] - 9s 4ms/step - loss:
0.0326
Epoch 16/30
```

```
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0324
Epoch 17/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0238
Epoch 18/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0227
Epoch 19/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0226
Epoch 20/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0224
Epoch 21/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0222
Epoch 22/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0226
Epoch 23/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0224
Epoch 24/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0226
Epoch 25/30
2012/2012 [==============================] - 9s 5ms/step - loss:
0.0226
Epoch 26/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0225
Epoch 27/30
2012/2012 [==============================] - 12s 6ms/step - loss:
0.0224
Epoch 28/30
2012/2012 [==============================] - 10s 5ms/step - loss:
0.0226
Epoch 29/30
2012/2012 [==============================] - 11s 6ms/step - loss:
0.0224
Epoch 30/30
2012/2012 [==============================] - 11s 5ms/step - loss:
0.0226

<keras.callbacks.History at 0x2cc7c880970>
```

```
dataset_val, capacity_val = load_data('B0006')
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity_val[attrib]
```

```
C = dis_ele['capacity'][0]
for i in range(len(dis_ele)):
    dis_ele['SoH']=(dis_ele['capacity']) / C
print(dataset_val.head(5))
print(dis_ele.head(5))
```

```
Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 2.035337591005598,
4.179799607333447, -0.0023663271409738672, 24.277567510331888, -
0.0006, 0.0, 0.0]
   cycle  ambient_temperature            datetime  capacity
voltage_measured  \
0      1                   24 2008-04-02 15:25:41  2.035338
4.179800
1      1                   24 2008-04-02 15:25:41  2.035338
4.179823
2      1                   24 2008-04-02 15:25:41  2.035338
3.966528
3      1                   24 2008-04-02 15:25:41  2.035338
3.945886
4      1                   24 2008-04-02 15:25:41  2.035338
3.930354

   current_measured  temperature_measured  current_load  voltage_load
time
0         -0.002366             24.277568       -0.0006         0.000
0.000
1          0.000434             24.277073       -0.0006         4.195
16.781
2         -2.014242             24.366226       -1.9990         3.070
35.703
3         -2.008730             24.515123       -1.9990         3.045
53.781
4         -2.013381             24.676053       -1.9990         3.026
71.922
   cycle            datetime  capacity       SoH
0      1 2008-04-02 15:25:41  2.035338  1.000000
1      2 2008-04-02 19:43:48  2.025140  0.994990
2      3 2008-04-03 00:01:06  2.013326  0.989185
3      4 2008-04-03 04:16:37  2.013285  0.989165
4      5 2008-04-03 08:33:25  2.000528  0.982898
```

A table is created containing the real SoH and the SoH predicted by the network and the root of the mean square error is calculated.

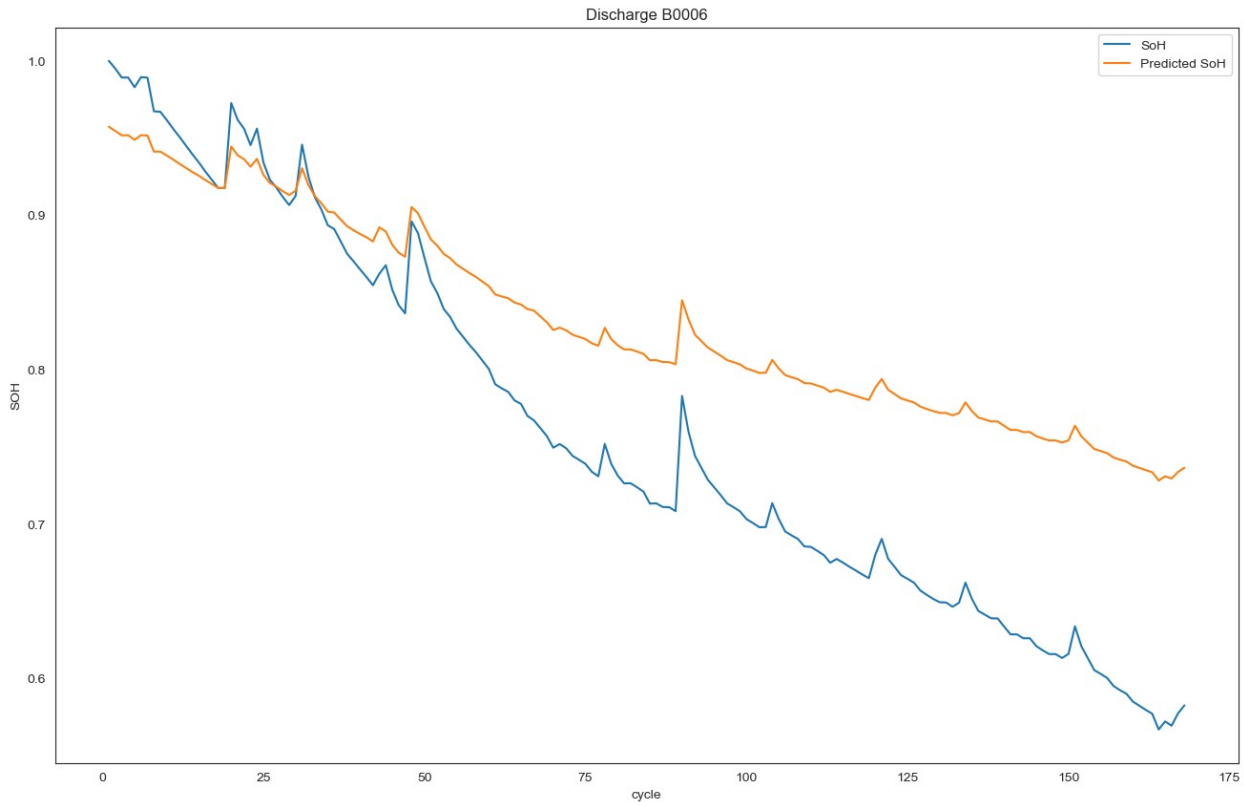
```
attrib=['capacity', 'voltage_measured', 'current_measured',
        'temperature_measured', 'current_load', 'voltage_load',
'time']
soh_pred = model.predict(sc.fit_transform(dataset_val[attrib]))
```

```
print(soh_pred.shape)

C = dataset_val['capacity'][0]
soh = []
for i in range(len(dataset_val)):
  soh.append(dataset_val['capacity'][i] / C)
new_soh = dataset_val.loc[(dataset_val['cycle'] >= 1), ['cycle']]
new_soh['SoH'] =  soh
new_soh['NewSoH'] = soh_pred
new_soh = new_soh.groupby(['cycle']).mean().reset_index()
print(new_soh.head(10))
rms = np.sqrt(mean_squared_error(new_soh['SoH'], new_soh['NewSoH']))
print('Root Mean Square Error: ', rms)
```

```
1572/1572 [==============================] - 2s 1ms/step
(50285, 1)
   cycle       SoH    NewSoH
0      1  1.000000  0.957288
1      2  0.994990  0.954583
2      3  0.989185  0.951744
3      4  0.989165  0.951791
4      5  0.982898  0.948813
5      6  0.989467  0.951850
6      7  0.989075  0.951582
7      8  0.967304  0.941209
8      9  0.966997  0.941175
9     10  0.961625  0.938682
Root Mean Square Error:  0.09164835812794936
```

```
plot_df = new_soh.loc[(new_soh['cycle']>=1),['cycle','SoH', 'NewSoH']]
sns.set_style("white")
plt.figure(figsize=(16, 10))
plt.plot(plot_df['cycle'], plot_df['SoH'], label='SoH')
plt.plot(plot_df['cycle'], plot_df['NewSoH'], label='Predicted SoH')
#Draw threshold
#plt.plot([0.,len(capacity)], [0.70, 0.70], label='Threshold')
plt.ylabel('SOH')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.legend()
plt.title('Discharge B0006')
```

```
Text(0.5, 1.0, 'Discharge B0006')
```




```
dataset_val, capacity_val = load_data('B0005')
attrib=['cycle', 'datetime', 'capacity']
dis_ele = capacity_val[attrib]
rows=['cycle','capacity']
dataset=dis_ele[rows]
data_train=dataset[(dataset['cycle']<50)]
data_set_train=data_train.iloc[:,1:2].values
data_test=dataset[(dataset['cycle']>=50)]
data_set_test=data_test.iloc[:,1:2].values

sc=MinMaxScaler(feature_range=(0,1))
data_set_train=sc.fit_transform(data_set_train)
data_set_test=sc.transform(data_set_test)

X_train=[]
y_train=[]
#take the last 10t to predict 10t+1
for i in range(10,49):
    X_train.append(data_set_train[i-10:i,0])
    y_train.append(data_set_train[i,0])
X_train,y_train=np.array(X_train),np.array(y_train)

X_train=np.reshape(X_train,(X_train.shape[0],X_train.shape[1],1))
```

```
Total data in dataset:  616
[1, 24, datetime.datetime(2008, 4, 2, 15, 25, 41), 1.8564874208181574,
4.191491807505295, -0.004901589207462691, 24.330033885570543, -0.0006,
0.0, 0.0]
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Masking,
TimeDistributed
from keras.models import Sequential
from keras.layers import Dense, Conv1D
from keras.layers import Dropout, Input
from keras import initializers
from keras.layers import MaxPooling1D
from keras.models import Model
from keras.layers import TimeDistributed, Flatten
from keras.layers.core import Dense, Activation, Dropout
from keras.callbacks import EarlyStopping
from keras.callbacks import ModelCheckpoint

input_shape=(X_train.shape[1],1)

model = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
input_shape=input_shape),
    Conv1D(filters=64, kernel_size=3, activation='relu'),
    LSTM(100, return_sequences=True),
    LSTM(100),
    Dense(1)
])
model.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error')

# Summary of the model architecture
model.summary()

# Summary of the model architecture
model.summary()
```

```
Model: "sequential_18"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d_28 (Conv1D)          (None, 8, 64)             256

 conv1d_29 (Conv1D)          (None, 6, 64)             12352

 lstm_32 (LSTM)              (None, 6, 100)            66000

 lstm_33 (LSTM)              (None, 100)               80400
```

```
 dense_24 (Dense)             (None, 1)                 101

=================================================================
Total params: 159,109
Trainable params: 159,109
Non-trainable params: 0
_________________________________________________________________
Model: "sequential_18"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv1d_28 (Conv1D)           (None, 8, 64)             256

 conv1d_29 (Conv1D)           (None, 6, 64)             12352

 lstm_32 (LSTM)               (None, 6, 100)            66000

 lstm_33 (LSTM)               (None, 100)               80400

 dense_24 (Dense)             (None, 1)                 101

=================================================================
Total params: 159,109
Trainable params: 159,109
Non-trainable params: 0
_________________________________________________________________
```

```
model.fit(X_train,y_train,epochs=10,batch_size=25)
```

```
Epoch 1/10
2/2 [==============================] - 3s 16ms/step - loss: 0.3333
Epoch 2/10
2/2 [==============================] - 0s 17ms/step - loss: 0.2437
Epoch 3/10
2/2 [==============================] - 0s 17ms/step - loss: 0.1433
Epoch 4/10
2/2 [==============================] - 0s 17ms/step - loss: 0.0555
Epoch 5/10
2/2 [==============================] - 0s 16ms/step - loss: 0.0707
Epoch 6/10
2/2 [==============================] - 0s 17ms/step - loss: 0.0778
Epoch 7/10
2/2 [==============================] - 0s 16ms/step - loss: 0.0405
Epoch 8/10
2/2 [==============================] - 0s 16ms/step - loss: 0.0393
Epoch 9/10
2/2 [==============================] - 0s 17ms/step - loss: 0.0472
Epoch 10/10
2/2 [==============================] - 0s 17ms/step - loss: 0.0488
```

```
<keras.callbacks.History at 0x2cea239e790>
```

```
print(len(data_test))
data_total=pd.concat((data_train['capacity'],
data_test['capacity']),axis=0)
inputs=data_total[len(data_total)-len(data_test)-10:].values
inputs=inputs.reshape(-1,1)
inputs=sc.transform(inputs)
```

```
119
```

```
X_test=[]
for i in range(10,129):
    X_test.append(inputs[i-10:i,0])
X_test=np.array(X_test)
X_test=np.reshape(X_test,(X_test.shape[0],X_test.shape[1],1))
pred=model.predict(X_test)
print(pred.shape)
pred=sc.inverse_transform(pred)
pred=pred[:,0]
tests=data_test.iloc[:,1:2]
rmse = np.sqrt(mean_squared_error(tests, pred))
print('Test RMSE: %.3f' % rmse)
metrics.r2_score(tests,pred)
```

```
4/4 [==============================] - 0s 6ms/step
(119, 1)
Test RMSE: 0.367
```

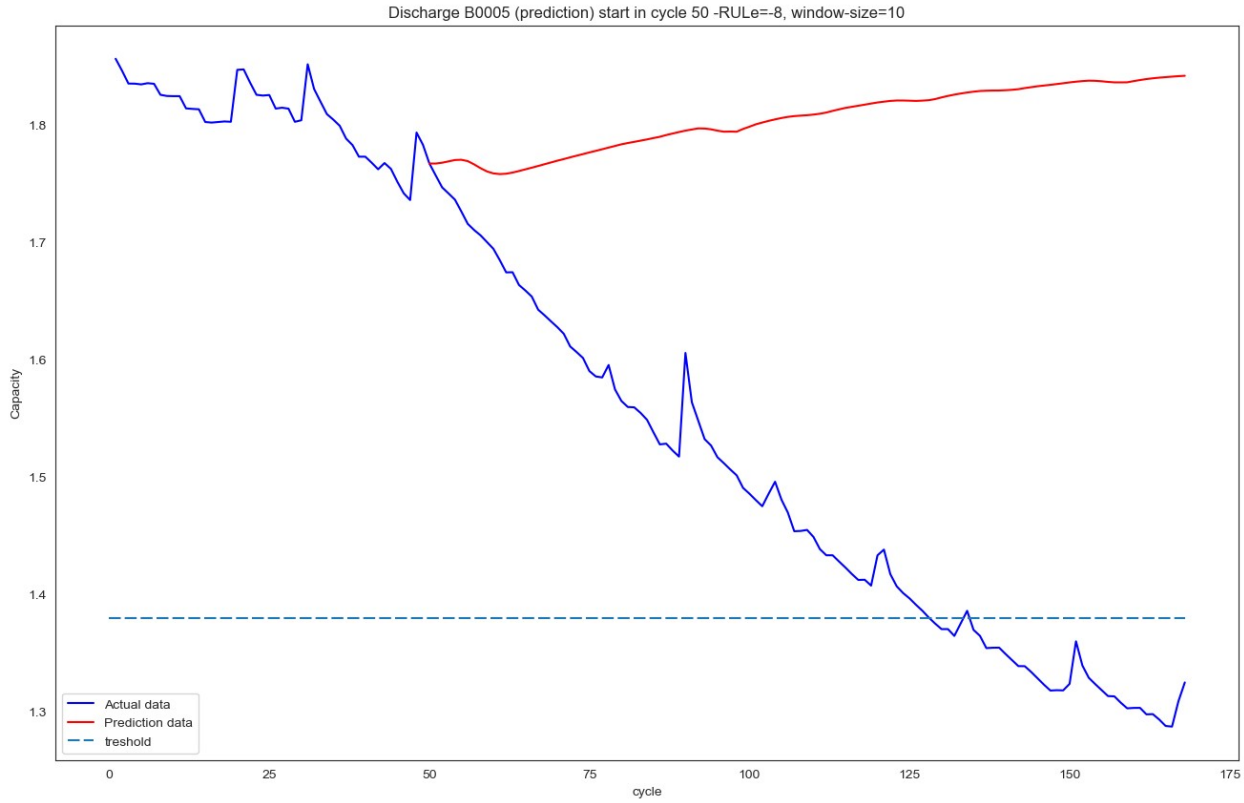
```
-6.352962516135343
```

As can be seen, the mean RMSE is 0.05 (5%), which is very close to the values observed in the literature using this type of network.

```
ln = len(data_train)
data_test['pre']=pred
plot_df = dataset.loc[(dataset['cycle']>=1),['cycle','capacity']]
plot_per = data_test.loc[(data_test['cycle']>=ln),['cycle','pre']]
plt.figure(figsize=(16, 10))
plt.plot(plot_df['cycle'], plot_df['capacity'], label="Actual data",
color='blue')
plt.plot(plot_per['cycle'],plot_per['pre'],label="Prediction data",
color='red')
#Draw threshold
plt.plot([0.,168], [1.38, 1.38],dashes=[6, 2], label="treshold")
plt.ylabel('Capacity')
# make x-axis ticks legible
adf = plt.gca().get_xaxis().get_major_formatter()
plt.xlabel('cycle')
plt.legend()
```

```
plt.title('Discharge B0005 (prediction) start in cycle 50 -RULe=-8,
window-size=10')
```

```
C:\Users\Kamal\AppData\Local\Temp\ipykernel_27760\1198164500.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  data_test['pre']=pred

Text(0.5, 1.0, 'Discharge B0005 (prediction) start in cycle 50 -RULe=-
8, window-size=10')
```



Finally, it can be seen in the graph that the capacity value and how it behaves over time is very close to the real value and supporting these data, the error in the estimation of the RUL was -8 which makes us understand that The model went ahead by 8 cycles to estimate that the battery reached its end of life.

```
pred=0
Afil=0
Pfil=0
a=data_test['capacity'].values
```

```
b=data_test['pre'].values
j=0
k=0
for i in range(len(a)):
    actual=a[i]

    if actual<=1.38:
        j=i
        Afil=j
        break
for i in range(len(a)):
    pred=b[i]
    if pred< 1.38:
        k=i
        Pfil=k
        break
print("The Actual fail at cycle number: "+ str(Afil+ln))
print("The prediction fail at cycle number: "+ str(Pfil+ln))
RULerror=Pfil-Afil
print("The error of RUL= "+ str(RULerror)+ " Cycle(s)")
```

```
The Actual fail at cycle number: 128
The prediction fail at cycle number: 49
The error of RUL= -79 Cycle(s)
```